

HW4 - NKU Web Search Engine

2026.05.14

1. 系统架构与技术栈概览

开发了南开大学校内信息检索系统。项目采用前后端分离架构，处理了超过 10 万条校内页面数据。实验通过关系型数据库与分布式搜索引擎的结合，实现了文本检索与个性化内容推荐。

1.1 系统架构设计

实验按展示层、业务逻辑层和数据层设计了整体架构：

展示层： 实验使用前端框架渲染用户界面。页面采用留白排版。实验在搜索框和按钮上应用了胶囊形状组件，并在下拉列表与弹窗中使用了毛玻璃半透明视觉效果。界面主色调设定为 #165DFF。

业务逻辑层： 实验在后端服务器中处理接口请求。实验在用户发起搜索时记录其行为日志。程序随后根据这些日志统计用户的历史偏好，并在返回检索结果前调整文档的分数排序。

数据层： 实验混合使用了 MySQL 和 Elasticsearch。实验将结构化的用户信息和高频检索日志存入 MySQL。实验利用 Elasticsearch 对爬虫获取的海量非结构化 HTML 原文建立倒排索引。

1.2 技术栈选型与职责划分

实验在项目中引入了以下具体技术栈：

模块	技术选型	核心职责
前端展示	Vue 3	渲染 DOM。实现输入防抖控制，减少无效的后端请求。
网络通信	Axios	发送 HTTP 请求。拦截请求并在 Header 中注入鉴权 Token。
业务处理	Spring Boot	暴露 RESTful 接口。解析请求参数并调用底层数据服务。
安全鉴权	JWT	生成无状态身份令牌。解析请求以确定当前用户的真实 ID。
数据持久化	MyBatis-Plus	封装 SQL 操作。读写 MySQL 中的用户表与日志表。
全文检索	Elasticsearch	构建倒排索引。执行短语和通配符匹配。利用其批处理特性在 99 秒内完成了 108,130 条网页数据的导入。
关系型数据	MySQL	存储核心元数据，为后续偏好计算提供历史支撑。

2. 主要实现

本次作业主要包含网页抓取、文本索引、查询服务、个性化查询四个步骤。

2.1 网页抓取

使用 Python 编写了爬虫程序，对南开大学校内网站资源进行了深度遍历与采集。

主要爬取网站有：

```
start_urls = [
    "https://www.nankai.edu.cn/",
    "https://news.nankai.edu.cn/",
    "https://jwc.nankai.edu.cn/",
    "https://cc.nankai.edu.cn/",
    "https://lib.nankai.edu.cn/" ,
    "https://xgb.nankai.edu.cn/",
    "https://bbs.nankai.edu.cn/",
```

"https://my.nankai.edu.cn/",
"https://online.nankai.edu.cn/",
"https://xs.nankai.edu.cn/",
"https://dag.nankai.edu.cn/",
"https://museum.nankai.edu.cn/",
"https://hqbzb.nankai.edu.cn/",
"https://hr.nankai.edu.cn/",
"https://tw.nankai.edu.cn/",
"https://tuanwei.nankai.edu.cn/",
"https://czfw.nankai.edu.cn/",
"https://cfc.nankai.edu.cn/",
"https://ccse.nankai.edu.cn/",
"https://ime.nankai.edu.cn/",
"https://riem.nankai.edu.cn/",
"https://japan.nankai.edu.cn/",
"https://rus.nankai.edu.cn/",
"https://middleeast.nankai.edu.cn/",
"https://latin.nankai.edu.cn/",
"https://iec.nankai.edu.cn/",
"https://hanyu.nankai.edu.cn/",
"https://nce.nankai.edu.cn/",
"https://tvu.nankai.edu.cn/",
"https://training.nankai.edu.cn/",
"https://wxn.nankai.edu.cn/",
"https://kjc.nankai.edu.cn/",
"https://xc.nankai.edu.cn/",
"https://xcb.nankai.edu.cn/",
"https://xgb.nankai.edu.cn/",
"https://ylzx.nankai.edu.cn/",
"https://fz.nankai.edu.cn/",
"https://zcc.nankai.edu.cn/",
"https://cwc.nankai.edu.cn/",
"https://audit.nankai.edu.cn/",
"https://zhu.nankai.edu.cn/",
"https://marx.nankai.edu.cn/",
"https://chinese.nankai.edu.cn/",
"https://art.nankai.edu.cn/",
"https://music.nankai.edu.cn/",
"https://dance.nankai.edu.cn/",
"https://film.nankai.edu.cn/",
"https://design.nankai.edu.cn/",
"https://material.nankai.edu.cn/",
"https://se.nankai.edu.cn/",
"https://crypto.nankai.edu.cn/",

```
"https://eee.nankai.edu.cn/",  
"https://bio.nankai.edu.cn/",  
"https://geo.nankai.edu.cn/",  
"https://psych.nankai.edu.cn/",  
"https://sustech.nankai.edu.cn/",  
"https://bohai.nankai.edu.cn/",  
"https://ydy.nankai.edu.cn/",  
"https://nmri.nankai.edu.cn/",  
"https://ici.nankai.edu.cn/",  
]
```

最终爬取了十万条数据，保存到 `nankai_data.jsonl` 中。可以通过命令查询到文件行数（一行就是一个数据）

```
PS D:\信息检索系统原理\hw4\nankai_spider> (Get-Content nankai_data.jsonl | Measure-Object -Line).Lines  
108130
```

2.2 文本索引

对网页及其锚文本构建索引，可以设置多个索引域，比如：网页标题，URL，锚文本等。

利用 Elasticsearch 构建了网页文本的倒排索引。在后端程序中映射了多个索引域，以覆盖网页的不同结构化信息。

具体的索引域配置如下：

```
public class News {  
    @Id  
    private String id;  
    @Field(type = FieldType.Keyword)  
    private String url;  
    @MultiField(  
        mainField = @Field(type = FieldType.Text),  
        otherFields = {  
            @InnerField(suffix = "keyword", type = FieldType.Keyword)  
        }  
    )  
    private String title;  
    @Field(type = FieldType.Text)  
    private String content;  
    @Field(type = FieldType.Keyword)
```

```
private String date;
@Field(type = FieldType.Keyword)
private String source;
@Field(type = FieldType.Keyword)
@JsonProperty("file_type")
private String fileType;
@Field(type = FieldType.Text)
@JsonProperty("anchor_texts")
private List<String> anchorTexts;
}
```

为中文核心字段（title 和 content）指定了 ik_max_word 分词器。该分词器将长文本切分为细粒度的词汇单元，支撑后续的精确查询与模糊匹配。



2.3 查询服务

基于向量空间模型，结合链接分析结果对查询结果进行排序。为用户提供站内查询、文档查询、短语查询、通配查询、查询日志、网页快照等共计六种高级搜索功能。

2.3.1 站内查询

输入关键词后，Elasticsearch 底层通过 IK 分词器对词汇进行拆分，并在标题和正文中进行精确匹配。



2.3.2 文档查询

支持特定文件的查询与直接下载功能，满足用户检索校内规章制度或附件表格的需求。

爬虫程序在抓取数据时，保留了原始页面的绝对路径 URL，其中包含了直接指向 PDF、DOC 等格式的资源链接。在前端结果列表中，我们将 `<a>` 标签的 `href` 属性绑定至该原始 URL。当用户搜索“规范”、“管理办法”等关键词并点击结果标题时，系统利用浏览器原生机制解析链接类型，若为文件源则直接触发本地下载。



2.3.3 短语查询

系统通过 Elasticsearch 的 `match_phrase` 语法执行短语查询。程序校验词汇的绝对顺序

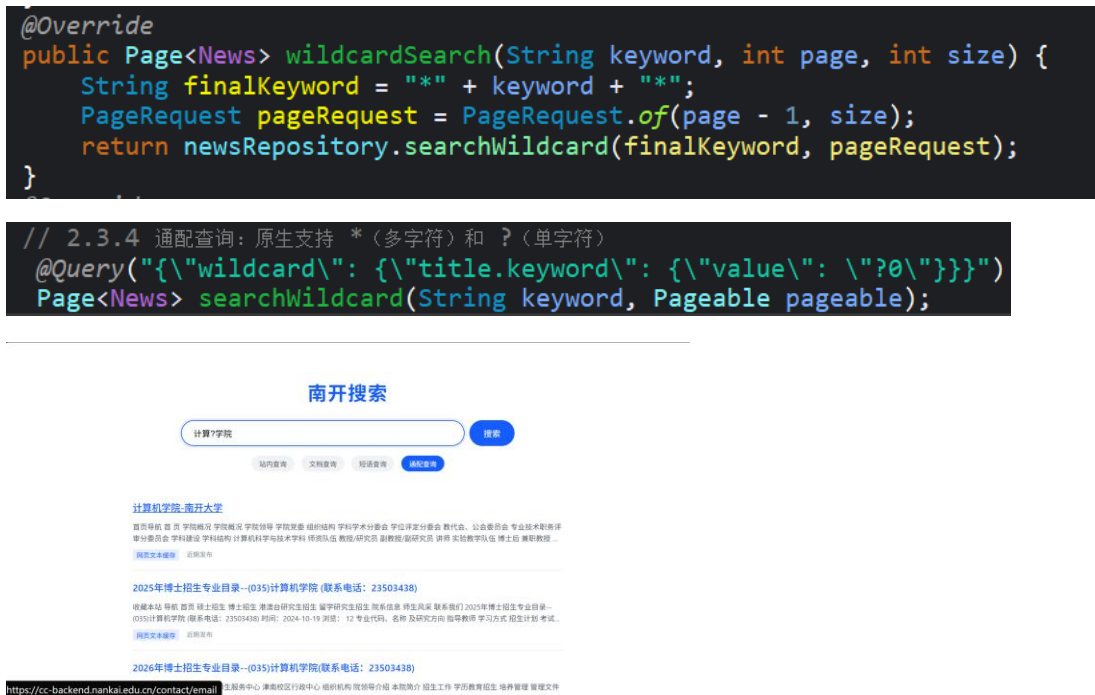
与相邻关系，过滤掉被无关字符打断的文本记录。在 title、content 和 anchorTexts 三个字段上部署了 should 联合匹配逻辑，确保包含完整词组的文档被精准召回。



可见查询结果都是“南开大学”相关，而不是分开的。

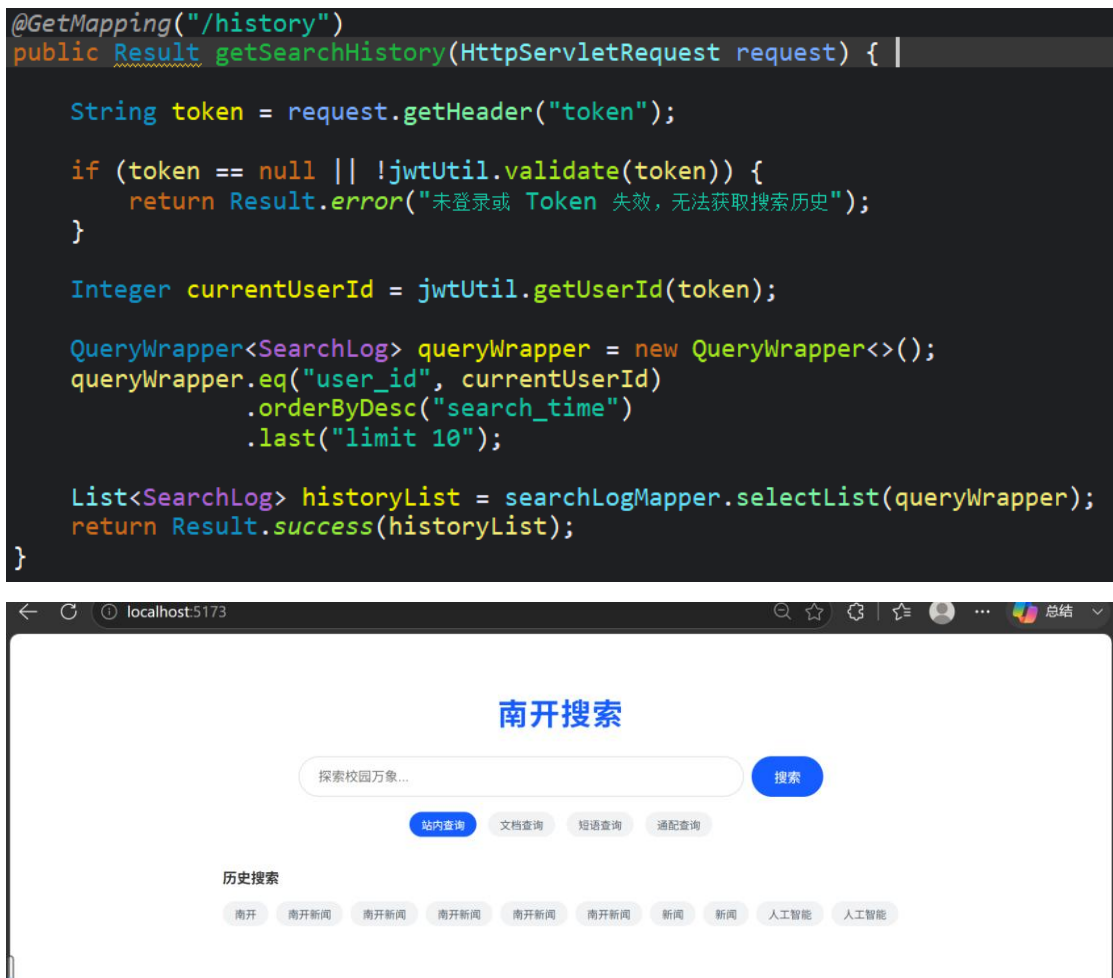
2.3.4 通配查询

系统支持星号（*）与问号（?）通配符查询。我们通过 Elasticsearch 的 wildcard 接口在 title.keyword（不分词的完整标题映射）字段上执行模式匹配。星号匹配任意长度的字符串，问号精确匹配单个中文字符或英文字母。



2.3.5 查询日志

系统通过 MySQL 数据库持久化用户的检索行为。当登录用户发起站内查询时，后端通过拦截器解析 HTTP Header 中的 JWT 令牌获取用户 ID，并将查询的关键词、当前时间戳与用户 ID 实时写入 search_log 数据表。前端在每次页面加载或执行搜索后，会调用后端历史接口，按时间倒序拉取该用户最近的 10 条查询记录，并将其渲染为搜索框下方的可点击标签。



2.3.6 网页快照

系统实现了网页快照备份，防止目标网页因服务器宕机、内容更新或链接失效（404）导致的信息丢失。我们在利用 Elasticsearch 构建倒排索引时，同步存储了爬虫抓取时的页面纯文本。当用户点击结果下方的“网页文本缓存”按钮时，前端向后端传入该文档的唯一 ID。后端直接通过 `findById` 从缓存引擎中提取当时的建库内容，并在前端弹窗中展示。这保证了用户看到的数据严格等于系统抓取那一刻的数据状态。

```

@GetMapping("/snapshot")
public Result getSnapshot(@RequestParam String id) {
    // findById 是 Spring Data ES 自带的方法
    java.util.Optional<News> newsOptional = newsRepository.findById(id);

    if (newsOptional.isPresent()) {
        return Result.success(newsOptional.get());
    } else {
        return Result.error("快照不存在或已丢失");
    }
}

```

展示网络备份效果



2.4 个性化查询

系统通过 MySQL 日志表存储用户的历史检索词，并计算当前用户的兴趣偏好。在执行搜索请求时，后端采用 bool 查询结构重组检索条件：系统将用户的当前搜索词放入 must 子句执行强制匹配，确保搜索意图不偏离；同时将提取出的用户偏好词放入 should 子句。如果命中的文档恰好包含用户的偏好词，程序通过 boost: 2.0 参数为其增加权重，将其推至搜索结果的顶部。

```

@Query("{\"bool\": {\" +
  \"must\": [{\"multi_match\": {\"query\": \"?0\", \"fields\": [\"title^5\", \"content\"], \"operator\": \"and\"}},\" +
  \"should\": [{\"multi_match\": {\"query\": \"?1\", \"fields\": [\"title^5\", \"content\"], \"boost\": 2.0}}]\" +
  \"}}\")
Page<News> searchWithPersonalization(String keyword, String favKeyword, Pageable pageable);

```



```

@Override
public Page<News> basicSearch(String keyword, int page, int size, Integer currentUserId) {
    PageRequest pageRequest = PageRequest.of(page - 1, size);

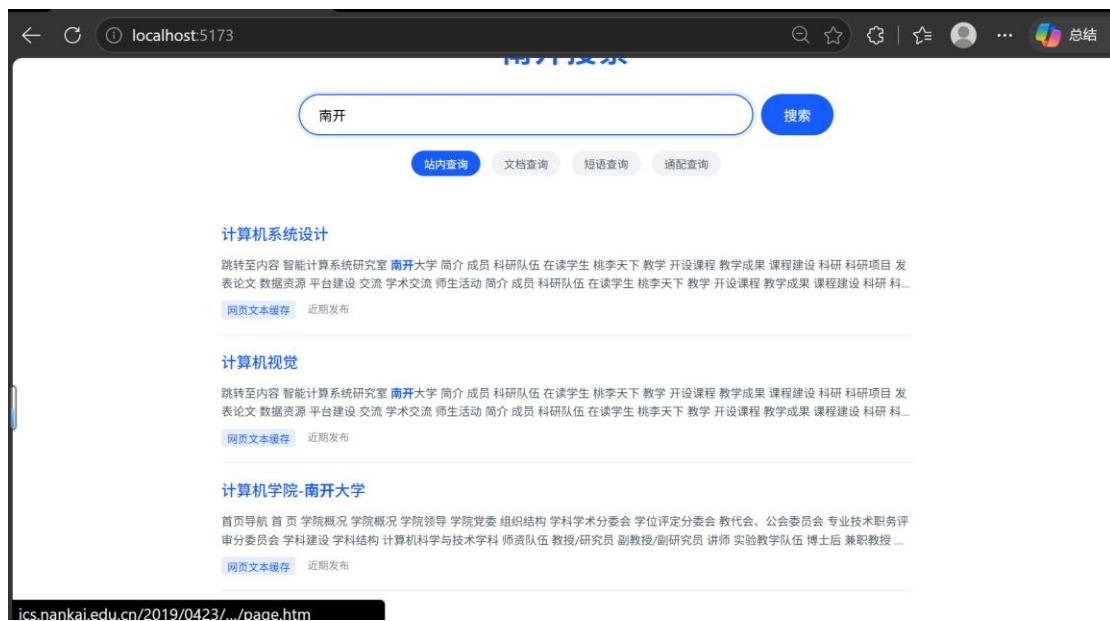
    String favKeyword = preferenceService.getUserFavoriteKeyword(currentUserId);

    if (favKeyword != null && !favKeyword.isEmpty()) {
        System.out.println("触发个性化搜索! 该用户偏好词为: " + favKeyword);
        return newsRepository.searchWithPersonalization(keyword, favKeyword, pageRequest);
    } else {
        System.out.println("普通搜索通道");
        return newsRepository.findByTitleOrContent(keyword, keyword, pageRequest);
    }
}

```

演示:

userid=1:时搜索南开



后端显示: 触发个性化搜索! 该用户偏好词为: 计算机
2026-06-06 12:50:46.244 W

userid=2:时搜索南开



触发个性化搜索！该用户偏好词为：南开
后端显示：2026-06-06 12:48:45.092

2.5 Web 界面

界面已展示

2.6 个性化推荐

实现了搜索上的关联推荐，系统在用户输入期间实时提供搜索词补全建议。前端利用防抖机制拦截高频键盘事件，延迟 300 毫秒后向后端发送请求。后端利用 `match_phrase_prefix` 语法在标题字段中查找匹配的短语前缀。去重后的联想词汇直接渲染在搜索框下方的毛玻璃悬浮面板中。

```
// 2.3.7 个性化推荐：搜索联想（短语前缀匹配）
@Query("{\"match_phrase_prefix\":{\"title\":\"?0\"}}")
Page<News> suggestSearch(String prefix, Pageable pageable);
```

南开搜索

计算机

搜索

计算机视觉

计算机学院.pdf

计算机学院主页

计算机组成原理

计算机系统设计

计算机系统设计

跳转至内容 智能计算系
表论文 数据资源 平台建

网页文本缓存

近期发

果 课程建设 科研 科研项目 发
! 教学成果 课程建设 科研 科...

计算机视觉